# The GUI folder — "Wellbeing Monitor" Streamlit dashboard

This document explains, in simple terms, how the `Scripts/GUI` folder works: what the dashboard shows on screen, what it lets you do, and — most importantly — where all the information it displays actually comes from inside the project.

The key idea is that the GUI does not contain any analysis logic of its own. It is only a control panel. Every number, every model, every overlay you see is produced by code that already exists elsewhere in the project (the Posture, Blinking, HeartRate and ML folders). The GUI simply reuses that code and shows the results in a browser.

---

## What the folder contains

The folder is small. There is the main application, a bridge file that connects the application to the rest of the project, a launch script, a theme configuration file and a short readme. The two files that matter are the application and the bridge: the application is everything you see and click, and the bridge is the single connection point to the rest of the project. The launch script just starts the dashboard (using the project's virtual environment if one exists), and the theme file gives the dashboard its green look and raises the maximum size for uploaded videos.

---

## The big picture: a panel on top of existing code

When the dashboard starts, the bridge file prepares everything so that the GUI can speak to the existing project code as if it were one program. It works out where all the project folders are, starting from its own location, and makes the analysis folders available so their modules can be loaded. Then it loads the project's real-time engine — the same engine the command-line scripts use — and takes from it the pieces the dashboard needs: the catalogue of available models, the object that trains and runs a model, the reader for the heart-rate wearable, and the routine that turns raw measurements into the row of numbers a model expects.

Because of this, the dashboard never reinvents anything. It runs the exact same code paths as the offline scripts. If the scripts and the GUI are given the same input, they produce the same result.

---

## What you see, and where it comes from

### The sidebar: all the choices

On the left there is a panel with every setting.

**The model.** You choose how many classes the model should predict (two classes, normal versus anomalous, or three classes, normal, sleepy or stress) and whether the model should be supervised or unsupervised. Based on these two choices, the list of available models fills in by itself. That list is not written into the GUI: it comes from the project's model catalogue, which in turn pulls the model

definitions from the machine-learning folder. So the menu always reflects the models that really exist in the project. For the two-class supervised case there is also an optional two-stage mode: first decide normal versus anomalous, and only if the result is anomalous, run a second model that says whether it is sleepy or stress.

When you press the train button, the chosen model is trained on the project's labelled data — the windows stored in the supervised data folder. The trained model is then kept in memory for the rest of the session, so it is not retrained on every interaction. If you later change one of the model choices, the dashboard notices that the active model no longer matches your selection and asks you to train again.

**The source.** You decide whether to analyse a live webcam or an uploaded video. For the webcam you can optionally enable the heart-rate wearable and tell it which serial port to use.

**The analysis window.** You set how long a window lasts, in seconds. A prediction is produced once per completed window. The default is sixty seconds, which matches the length of the windows the models were trained on; a shorter value just gives quicker results during a demo.

**The thresholds.** These sliders set the posture and blink thresholds. Their starting values are not hard-coded into the GUI: they are read straight from the project's posture and blink configuration files, so the panel always shows the real, literature-based values used by the offline analysis. When you start an acquisition, the values from these sliders are written back into those same configuration modules, which the analysers then read. This is why moving a slider changes the live analysis without anyone editing a script file.

## The central area: the live view and the results

In the middle you see the camera or video image with two kinds of overlay drawn on top: the body skeleton and the boxes around the eyes. These overlays are drawn by the project's shared drawing module, using the body landmarks and the eye points produced by the analysers.

Above or beside the image there is a large coloured banner showing the current state — normal, sleepy, stress or anomalous — and, when the model provides them, the probabilities for each class. The colour changes with the state. This banner shows the latest prediction the model produced.

There is also a small panel of live readings: how long until the next prediction, the current eye measurement, whether posture is being tracked, the wearable's current heartbeat (or a note that it is not connected), and how many samples have been collected so far in the current window.

Below the image there are live charts, one per signal, because the signals live on different scales. They show the eye measurement, the posture angles, the forward-head measure and, if available, the heart rate. These charts are fed continuously while the analysis runs, but at a reduced rate so the interface stays light: the model only outputs a label once per window, but the underlying signals are measured on every frame, and a sample of them is added to the charts at a steady pace.

At the bottom, after each acquisition, there are two tables. One is the log of predictions, one row per window, with a small summary of how often each state occurred. The other lists every feature that was computed for each window, grouped by signal. These tables are filled from what the dashboard recorded during the run.

---

## How a single run actually works

Whether the source is the webcam or a video, the rhythm is the same.

For every frame, the dashboard does the same handful of things. It detects the body and reads the posture values from it. It analyses the face to get the eye measurement and whether the eye is closed. It collects these into running buffers. It draws the overlays on the image and, every so often, adds a point to the live charts. In the webcam case it also keeps checking the wearable in the background; in the video case it instead looks up the heart-rate value for the current minute from the optional uploaded file.

None of this produces a prediction yet. A prediction only happens when a window finishes — every sixty seconds by default, or once per minute for a video. At that moment the buffered posture and blink measurements are summarised, the heart-rate values for the window are taken (from the wearable or the file), and all of it is combined into a single row of eighteen numbers. That row is the only thing the model ever sees. The model turns it into a state, and that state goes three places at once: the prediction log, the per-window feature table, and the coloured banner. Then the buffers are cleared and the next window begins.

The video case adds only two differences: a progress bar, and the fact that the heart-rate numbers come from the uploaded file (one row per minute, lined up with the video's minutes) instead of from a live device. Otherwise the analysis is identical, which is the whole point — the same pipeline drives both.

For the webcam, stopping works through the dashboard's natural refresh: when you press stop, the loop is interrupted at the next screen update and the page is redrawn in its stopped state, with the last run's charts still visible.

---

## The eighteen numbers the model sees

Everything funnels down to one row of eighteen values per window. Nine of them describe posture (the shoulder, head-tilt and forward-head measures, each as an average, a variability and a share of "bad" time). Seven describe blinking (the eye-openness average and variability, the closed-eye proportion, the blink count and rate, and the timing between blinks). The last two are the heart rate and a heart-rate-variability measure.

This single row is the only input the models use, and it is built the same way for the offline scripts and for the dashboard. That is what keeps training and live prediction consistent: the model is fed exactly the kind of row it was trained on.

---

## What happens if one of the three signals is missing

The three signals — posture, blinking and heart rate — are not treated equally when one of them is unavailable.

The heart rate is effectively optional. If the wearable is not connected, or no heart-rate file is provided for a video, the row of eighteen numbers is still built, with the two heart values left empty. The models can cope with this on their own, because every model begins by filling in missing values with the median value seen during training. So the prediction still happens; the heart part is simply imputed. This is by design.

Posture and blinking are different. If, for most of a window, the body cannot be found or the face and eyes cannot be measured, there is nothing to summarise for that signal, and the row of eighteen numbers cannot be assembled. In that case the dashboard does not invent a prediction: it simply skips that window. Nothing crashes — the banner keeps showing the previous state, no row is added to the log, and as soon as the person is back in view the predictions resume.

So the short answer is: a missing heart rate does not stop anything, the code keeps running and the model fills the gap. A missing posture or blinking signal does not crash anything either, but it means that particular window produces no prediction until the signal comes back.

---

## Common problems and what they mean

If the webcam cannot be opened, it is usually because another application is using the camera, the camera permission has not been granted, or the dashboard is running on a machine without a camera. The webcam is local, so the dashboard has to run on the machine the camera is attached to.

If the banner stays on "waiting", it means no window has finished yet, or the person was out of frame for most of the window so no prediction could be made. Staying in view, with decent lighting, and waiting for a full window to complete fixes this; shortening the window length makes it happen sooner.

If the train button is disabled, there is no model available for the chosen combination of settings — some unsupervised combinations leave out models that cannot label new samples. Changing the class or learning choice brings models back.

If training fails saying a file is missing, the labelled training data has not been generated yet and needs to be created before a model can be trained.

If the dashboard warns that the selection changed, it means a model setting was changed after training and the model needs to be retrained to match.

If the wearable shows as not connected and the heart values are empty, the device is absent, on the wrong port, or its driver is missing. This is not fatal: the heart values are imputed by the model.

If an uploaded heart file cannot be read, or its columns are not recognised, the file is ignored and the heart values are imputed. The file should have a heart-rate column and a variability column, one row per minute, lined up with the video's minutes.

If a video cannot be opened, the format is not supported; a common format should be used instead.

If changing a threshold slider seems to have no effect, it is because the settings are applied when an acquisition starts. Thresholds should be set before pressing start or analyse; for a run already in progress, stop it and start again.

Finally, if the charts and log disappear between runs, that is intentional: clearing the results, or starting a new video, resets them so old data is not mixed into the next acquisition.